

데이터 분석 7강

머신러닝 기초 개념

지도학습 · 비지도학습 · 훈련/테스트 · sklearn 파이프라인

알고리즘을 배우기 전, 반드시 알아야 할 머신러닝의 기본 개념

 지도학습

 비지도학습

 sklearn

오늘의 학습 목표

오늘 수업이 끝나면 — 머신러닝이 어떻게 동작하는지 큰 그림을 그릴 수 있습니다

1

지도 vs 비지도

레이블(정답) 유무로 분류

분류/회귀 = 지도학습

군집화 = 비지도학습

각각의 대표 사용 사례 설명

2

훈련/테스트 분리

왜 데이터를 나눠야 하는가

과적합(Overfitting) 이해

train_test_split 사용법

x(특성) / y(타겟) 구분

3

sklearn 파이프라인

5단계 기본 흐름 암기

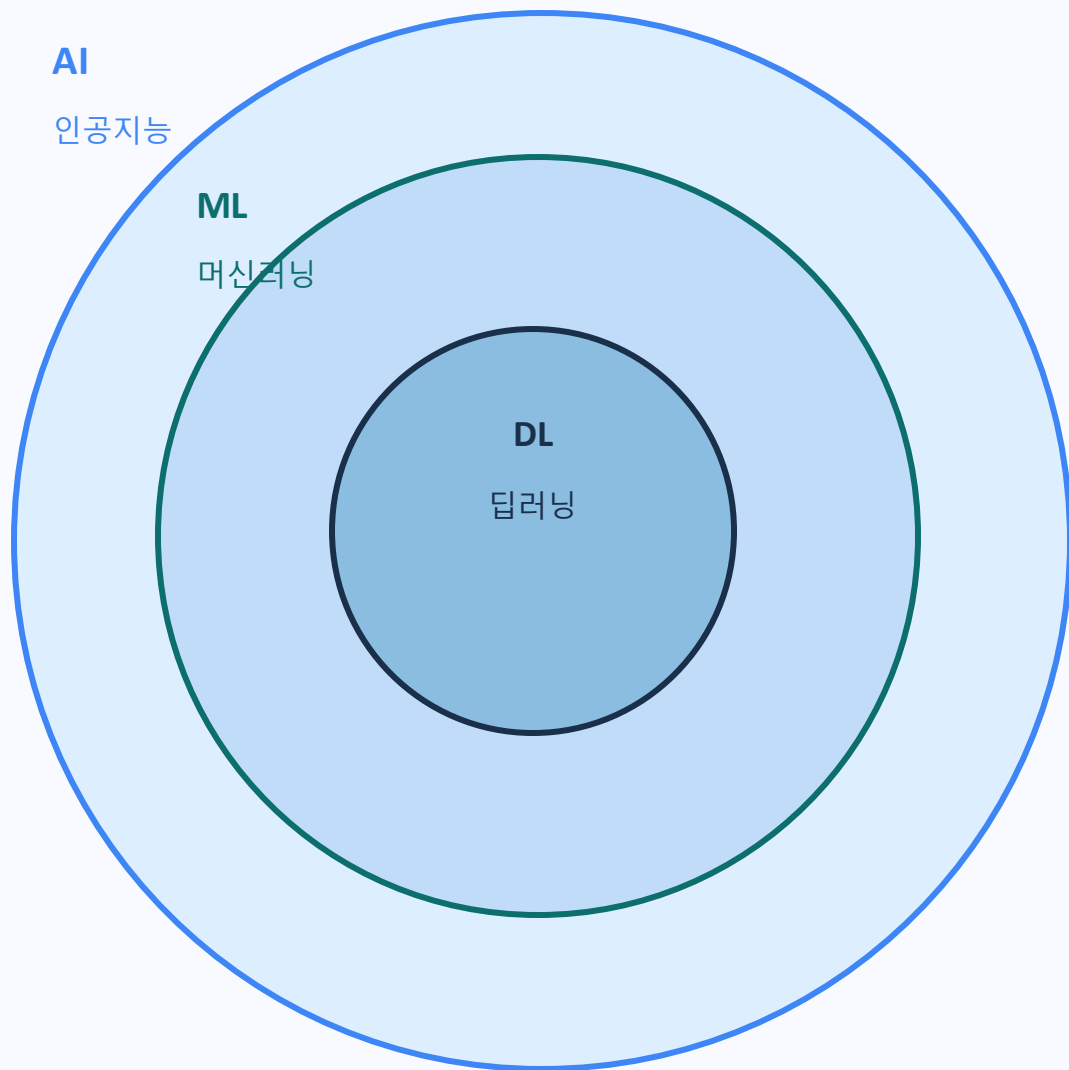
fit / predict / score 역할

전처리 도구 2가지

첫 모델 직접 실행

 구성 : PART 1 (30분) + PART 2 (30분) + PART 3 (20분) + 실습 (30분) + 정리 (10분) = 2시간

머신러닝이란? — AI의 한 분야



💡 머신러닝 핵심 개념

기존 프로그래밍

사람이 규칙을 직접 작성 → 컴퓨터가 따름

머신러닝

데이터에서 컴퓨터가 스스로 패턴·규칙을 학습

Why ML?

규칙이 너무 복잡하거나
데이터가 너무 많을 때 인간이 직접 규칙을 만들기 어려움

실생활 예시

이메일 스팸 필터 · 유튜브 추천
얼굴 인식 · 신용카드 이상 탐지



PART 1

지도학습 vs 비지도학습

지도학습 (Supervised Learning)

🔑 레이블(정답)이 있는 데이터로 모델을 학습시켜 새 데이터의 정답을 예측하는 방식

📖 훈련 데이터

키(cm)	몸무게(kg)	성별
170	65	남 ✓
160	52	여 ✓
175	72	남 ✓
155	48	여 ✓

✓ 정답(레이블) 포함



🐼
학습된
모델



🔍 새 데이터 → 예측

키: 168cm
몸무게: 60kg
성별: ???

예측: 남 (78% 확률)

🔴 분류 (Classification)

타깃이 범주형

예: 스팸/정상, 합격/불합격, 품종

📈 회귀 (Regression)

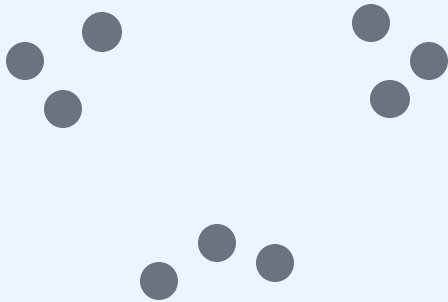
타깃이 연속 수치형

예: 집값 예측, 시험 점수 예측

비지도 학습 (Unsupervised Learning)

🔍 정답(레이블)이 없는 데이터에서 숨겨진 패턴·구조를 스스로 찾아내는 방식

📄 레이블 없는 데이터



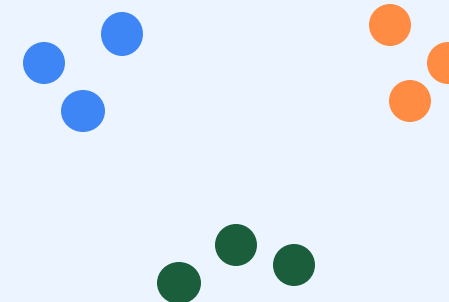
레이블 없음 😊



비지도
알고리즘



🎯 군집화 결과



자동으로 그룹 발견 🎯

🎯 군집화 (Clustering)

비슷한 데이터끼리 그룹 분류
예: 고객 세분화, 뉴스 토픽 분류

📉 차원 축소 (Dimensionality Reduction)

고차원 데이터를 저차원으로 압축·시각화
예: PCA — 주성분 분석

🚨 이상 탐지 (Anomaly Detection)

평균과 크게 다른 이상한 데이터 포인트 발견
예: 금융 사기 탐지, 불량품 검출

지도학습 vs 비지도학습 — 한눈에 비교

비교 항목	지도학습	비지도학습
레이블(정답)	필요 ☒	불필요 ✕
목적	예측·분류	패턴 발견
대표 유형	분류 / 회귀	군집화 / 차원 축소
대표 알고리즘	결정 트리, KNN, 선형 회귀	K-Means, PCA
실생활 예시	스팸 필터, 집값 예측, 진단	고객 세분화, 뉴스 클러스터링
데이터 준비 난이도	높음 (레이블 수작업 필요)	낮음 (데이터만 있으면 됨)

💡 이 수업에서는?

지도학습(분류/회귀)은 9-10강에서 별도로 다룹니다. 오늘은 개념과 구분만! 비지도학습(군집화)은 11강에서 다룹니다.



PART 2

훈련 데이터 vs 테스트 데이터

왜 데이터를 나눠야 할까? — 과적합(Overfitting)

☐ 시험 문제집을 통째로 외운 학생 — 문제집은 100점, 실제 시험은 50점?

✗ 나쁜 방법

전체 데이터로 훈련 후 같은 데이터로 평가

훈련 데이터 100개

모델 학습 → 같은 100개로 테스트

정확도: 98% (착각!)

👤 실제 새 데이터에서는 형편없을 수 있음

✓ 좋은 방법

훈련/테스트 데이터를 미리 분리

전체 100개 → 훈련 80개 + 테스트 20개

훈련 80개로 학습
→ 본 적 없는 20개로 평가

✓ 실제 성능을 정직하게 측정

☐ 과적합이란?

훈련 데이터에만 과도하게 최적화된 상태

훈련 정확도: 99%
테스트 정확도: 55%

둘의 차이가 크면
= 과적합(Overfitting) 의심!

⚠ 일반화 능력이 없는 모델 → 쓸모없음

결론: 모델이 "처음 보는 데이터"에서도 잘 동작하는지 확인하기 위해 테스트 세트는 절대 훈련에 사용하지 않습니다

train_test_split — 데이터 분리 실전

전체 데이터 (100개)

↓ test_size=0.2 (80%/20%)

🐾 훈련 세트 (Train Set) — 80개

🐾 테스트 20개

```
from sklearn.model_selection import train_test_split

# X: 특성 (Feature), y: 타겟 (Target)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20%를 테스트 세트로
    random_state=42     # 재현성을 위한 난수 고정
)

print(X_train.shape)   # (80, n_features)
print(X_test.shape)    # (20, n_features)
```

test_size

= 0.2

테스트 비율 (0.2 = 20%)
일반적으로 0.2 ~ 0.3 사용

random_state

= 42

난수 시드 고정
같은 숫자 = 항상 같은 결과

shuffle

= True

기본값 True → 섞은 후 분리
시계열 데이터는 False 권장

stratify

= y

클래스 비율 유지하며 분리
불균형 데이터에 유용

Feature(X) vs Target(y) — 입력과 출력 구분

X Feature (특성)

모델에 입력하는 변수들
= 독립 변수 = 입력값
예: 키, 몸무게, 나이, 성별

```
# DataFrame에서 X 만들기
X = df[['키', '몸무게', '나이']] # 여러 열
# 또는
X = df.drop('합격', axis=1)      # 타겟 빼고 전부
```

 shape: (100, 3) → 100개 행, 3개 특성

y Target (타겟)

모델이 예측해야 할 변수
= 종속 변수 = 출력값
예: 합격 여부, 집값

```
# DataFrame에서 y 만들기
y = df['합격여부'] # 단일 열

# y는 보통 Series 형태
print(y.dtype)    # object or int64
```

 shape: (100,) → 100개 정답값

💡 관례: 대문자 X = 2차원 행렬(DataFrame), 소문자 y = 1차원 벡터(Series). sklearn의 모든 함수가 이 규칙을 따릅니다.

모델 평가 기초 — 성능은 어떻게 측정할까?

모델이 "얼마나 잘 예측했는지"를 숫자로 표현 — 학습 데이터가 아닌 테스트 데이터로 측정!

● 분류 (Classification) 평가 지표

정확도 (Accuracy)

정답 개수 / 전체 개수 | 가장 기본, 불균형 데이터엔 주의

정밀도 (Precision)

$TP / (TP + FP)$ | "양성 예측이 얼마나 맞았나?"

재현율 (Recall)

$TP / (TP + FN)$ | "실제 양성을 얼마나 잡았나?"

F1-Score

정밀도·재현율의 조화평균 | 불균형 데이터 종합 점수

☑ 회귀 (Regression) 평가 지표

MAE

평균 절대 오차 | 예측값과 실제값 차이의 평균

MSE

평균 제곱 오차 | 큰 오차에 더 큰 패널티

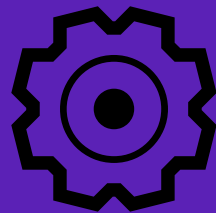
RMSE

MSE의 제곱근 | 같은 단위로 해석 가능

R^2 (결정계수)

0~1, 1에 가까울수록 좋음 | "얼마나 잘 설명하나?"

🔗 오늘은 `score()` 함수(정확도)만 사용합니다. 정밀도/재현율 등 상세 지표는 분류 강의(9강)에서 자세히 배웁니다.



PART 3

sklearn 기본 파이프라인

sklearn 5단계 파이프라인 — 모든 모델에 공통 적용

1

 데이터 준비

x(특성), y(타겟) 분리
train_test_split으로 훈련/테스트 나누기

2

 전처리

스케일링, 인코딩, 결측치 처리
(StandardScaler, LabelEncoder)

3

 모델 선택

알고리즘 선택 및 인스턴스 생성
model = DecisionTreeClassifier()

4

 학습 (fit)

훈련 데이터로 모델 학습
model.fit(X_train, y_train)

5

 예측/평가

테스트 데이터로 예측 및 점수 확인
model.predict() model.score()

fit / predict / score — 3가지 핵심 메서드

.fit()

학습 (훈련 데이터 전용)

모델이 훈련 데이터에서
패턴·규칙을 학습합니다
테스트 데이터로는 절대 fit() 금지!

```
model.fit(X_train, y_train)
```

비유 🔍

요리사가 레시피 외우는 것

.predict()

예측 (새 데이터 입력)

학습된 모델로
새 x값에 대한 y를 예측합니다
결과는 numpy 배열로 반환

```
y_pred = model.predict(X_test)
```

비유 🔍

외운 레시피로 새 요리

.score()

평가 (정확도 반환)

분류: 정확도 (0.0 ~ 1.0)
회귀: R^2 결정계수
내부적으로 predict() 후 비교

```
acc = model.score(X_test, y_test)
```

비유 🔍

손님이 맛을 평가

```
model.fit(X_train, y_train)    # 1. 학습
y_pred = model.predict(X_test) # 2. 예측
acc = model.score(X_test, y_test) # 3. 평가
print(f"정확도: {acc:.2%}")
```

전처리 — 모델 학습 전 데이터 다듬기

머신러닝 모델은 숫자만 이해합니다 → 범주형 데이터는 숫자로, 크기가 다른 수치는 비슷한 범위로 맞춰줘야 합니다



StandardScaler

수치형 데이터 스케일링

⚠ 왜 필요? 키(170), 나이(17), 소득(3000만)처럼 단위가 다르면 큰 숫자가 모델을 지배함

✓ 효과 각 특성의 평균=0, 표준편차=1로 표준화하여 동등한 영향력 부여

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test) # fit 금지!
```

⚠ test에는 transform()만! fit_transform() 사용 금지



LabelEncoder

범주형 데이터 → 숫자 변환

⚠ 왜 필요? '남'/'여', '서울'/'부산' 같은 텍스트는 모델이 처리할 수 없음

✓ 효과 '남'→0, '여'→1 처럼 문자열을 정수 레이블로 변환

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_enc = le.fit_transform(y) # '남'→0, '여'→1
print(le.classes_)         # ['남' '여']
```

⚠ 타겟 y에 사용. 다중 범주는 OneHotEncoder 권장

실습 ① — 데이터 분리 직접 해보기

1

미션 1 데이터 준비 & 분리

🕒 5분

- 1 student_score.csv 불러오기
- 2 x = 수학/영어/과학점수 3열 선택
- 3 y = 합격여부 열 선택
- 4 train_test_split(test_size=0.2, random_state=42)
- 5 X_train.shape, X_test.shape 출력 확인

2

미션 2 스케일링 적용

🕒 5분

- 1 StandardScaler 임포트
- 2 scaler.fit_transform(X_train) → X_train_s
- 3 scaler.transform(X_test) → X_test_s
- 4 X_train_s[0] 출력 → 값 범위 확인
- 5 합격여부 컬럼 LabelEncoder 적용

```
X = df[_____] # ① 특성 3열
y = df['_____'] # ② 타겟 열

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=_____, random_state=42)
X_train_s = scaler._____(X_train) # ③ fit+transform
X_test_s = scaler._____(X_test) # ④ transform only
```

실습 ② — sklearn 첫 모델 만들고 평가하기

1

미션 1 모델 학습

🕒 5분

- 1 DecisionTreeClassifier 임포트
- 2 model = DecisionTreeClassifier()
- 3 model.fit(X_train_s, y_train_enc)
- 4 fit 완료 후 모델 객체 출력

2

미션 2 예측 & 비교

🕒 5분

- 1 y_pred = model.predict(X_test_s)
- 2 y_test_enc[:10] vs y_pred[:10] 비교
- 3 틀린 것 몇 개인지 직접 세기
- 4 (y_pred != y_test_enc).sum() 확인

3

미션 3 평가 & 토론

🕒 5분

- 1 model.score(X_test_s, y_test_enc)
- 2 정확도가 얼마나 나왔나요?
- 3 random_state 바꾸면 어떻게 될까?
- 4 합격 기준을 바꾸면 성능 변화?

```
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

model = DecisionTreeClassifier()
model.fit(X_train_s, _____) # ①
y_pred = model.predict(_____) # ②
acc = model.score(_____, _____) # ③
print(f"정확도: {acc:.2%}")
```

7강 핵심 정리 — 오늘 배운 것을 기억하세요

1

 지도 vs 비지도

지도학습 = 레이블(정답) 있음 → 예측·분류

비지도학습 = 레이블 없음 → 패턴 발견

분류 / 회귀 = 지도학습의 두 종류

군집화 / 차원축소 = 비지도학습의 예

2

 훈련/테스트 분리

테스트 데이터 = 절대 훈련에 쓰지 않는다

과적합 = 훈련 정확도 높고 테스트 낮음

`train_test_split(test_size=0.2, random_state=42)`

x = 특성(Feature), y = 타겟(Target)

3

 sklearn 파이프라인

5단계: 준비 → 전처리 → 모델 → 학습 → 평가

`fit()` = 학습, `predict()` = 예측, `score()` = 평가

StandardScaler: test에는 `transform()`만 적용

LabelEncoder: 범주형 → 정수 변환

☆ 암기 공식

`fit(훈련)` → `predict(예측)` → `score(평가)` | 테스트에는 절대 `fit()` 사용 금지 | x 는 2D, y 는 1D

수고하셨습니다! 🎉

7강 머신러닝 기초 — 완료

✓ 오늘 배운 것

- ✓ 지도학습 vs 비지도학습 구분
- ✓ 훈련/테스트 분리 이유와 과적합
- ✓ X(특성) vs y(타겟) 개념
- ✓ sklearn 5단계 파이프라인
- ✓ fit / predict / score 3메서드
- ✓ 전처리: StandardScaler, LabelEncoder

→ 8강 예고

선형 회귀	집값·점수 예측하는 첫 번째 모델
단순/다중 회귀	특성이 1개 vs 여러 개일 때
OLS와 경사하강법	모델이 학습하는 원리
회귀 평가 지표	MAE, MSE, R^2 실습으로 이해